

ARTIFICIAL INTELLIGENCE IN FINTECH / WEEK 08

Building an Equal-Weighted Portfolio

Equal weighting assigns the same fraction of invested money to each stock. We use one four-stock example to connect weights, shares, price changes, rebalancing and fees. Then we examine two limits: equal money is not equal risk, and adding a stock does not necessarily lower volatility. We do not select another weighting method.

Issue	Question	Page
1	Does equal weighting mean equal numbers of shares?	3
2	How do four stock returns become one portfolio return?	4
3	Why do equal weights change without trading?	5
4	What changes when only whole shares can be bought?	6
5	When is a monthly rebalance executed?	7
6	Which holdings must be bought or sold to restore 25%?	8
7	Which traded dollars receive the cost charge?	9
8	How are fees paid before equal allocation?	10-12
9	How do we follow the account between rebalances?	13
10	Does rebalancing more often leave more money?	14
11	Do equal dollar allocations have equal risk?	15
12	Does adding a stock necessarily reduce volatility?	16

Use the examples in order. Each issue ends with two discussion questions, not a separate submission. Market observations and teaching assumptions are identified separately.

SHARED DATA AND SETTINGS

Prepare the common example

Use AAPL, MSFT, JPM and XOM, January 2-June 28, 2024, matching the earlier evaluation period. Prices are provider Close in USD/share, not dividend-adjusted total returns. Capital is USD 1,000. The stocks and rules are teaching choices, not historically documented investment decisions.

Before running: Will all four stocks have prices on the same first and last dates?

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
tickers = ["AAPL", "MSFT", "JPM", "XOM"]
history = {}
for ticker in tickers:
    h = yf.Ticker(ticker).history(start="2023-12-29",
                                end="2024-07-01", auto_adjust=False, actions=True)
    h.index = h.index.strftime("%Y-%m-%d")
    assert h.index.is_unique and h.index.is_monotonic_increasing
    assert h["Stock Splits"].eq(0).all()
    history[ticker] = h
all_close = pd.DataFrame({t: h["Close"] for t, h in history.items()})
assert np.isfinite(all_close).all().all() and all_close.gt(0).all().all()
close = all_close.loc["2024-01-02":].copy()
returns = close.pct_change(fill_method=None).dropna()
shown = close.iloc[[0, -1]].round(2)
shown.insert(0, "Date", shown.index)
shown
```

Date	AAPL	MSFT	JPM	XOM
2024-01-02	185.64	370.87	172.08	102.36
2024-06-28	210.62	446.95	202.26	115.12

The request returns 124 common prices per stock and 123 daily return intervals. December 29 is used only to label the first preceding decision date. No price is filled or interpolated.

Assume fractional shares and fills at the displayed Close; these are not observed executions. Start at the January 2 Close. Monthly trades occur at the first observed Close of each new month. No final-Close rebalance or liquidation. Exclude dividends, taxes and cash interest. Issues 1-6 omit fees; Issues 7-10 introduce an assumed 4 bp charge on every dollar bought or sold.

The monthly schedule is our teaching rule, not a reproduction of the MSCI index. Week 6 used a linear cost approximation; here explicit share and cash accounting pays fees from the account. Comparisons within this week use that same accounting.

ISSUE 01

Does equal weighting mean equal numbers of shares?

Equal weighting divides money, not share counts. With four stocks, each receives 25% of the invested amount. Different prices therefore require different numbers of shares.

N is the number of stocks; w_i is a target weight (a fraction). W is money to invest (USD), C_i is the January 2 Close (USD/share), and q_i is shares. Fees are zero here.

$$w_i = \frac{1}{N}, \quad q_i = \frac{w_i W}{C_i}$$

$$w_i = \frac{1}{4} = 0.25, \quad q_{\text{AAPL}} \approx \frac{0.25 \times 1000}{185.64000} \approx 1.34669 \text{ shares}$$

Before running: Will the highest-priced stock require more shares or fewer shares for the same USD 250?

```
capital = 1000.0
weight = 1 / len(tickers)
shares = capital * weight / close.iloc[0]
pd.DataFrame({"Stock": tickers, "Close (USD)": close.iloc[0].round(2),
             "Invested (USD)": capital * weight,
             "Shares": shares.round(5), "Weight (%)": 100 * weight})
```

Stock	Close (USD)	Invested (USD)	Shares	Weight (%)
AAPL	185.64	250.0	1.34669	25.0
MSFT	370.87	250.0	0.67409	25.0
JPM	172.08	250.0	1.45281	25.0
XOM	102.36	250.0	2.44236	25.0

MSFT costs USD 370.87/share, so USD 250 buys approximately 0.67409 shares. XOM costs USD 102.36/share and requires 2.44236 shares. Both positions have weight 25%; buying one share of each would not.

Discuss

1. Using these four prices, calculate the weights if one share of each stock is bought. Which stock has the largest weight?
2. If capital doubles with the same prices and no fees, which of the displayed quantities double, and which remain unchanged?

Sources: [1], [2], [3]. See the references page.

ISSUE 02

How do four stock returns become one portfolio return?

A portfolio return measures the change in the whole account. Add the stock profits, then divide by starting wealth. Equivalently, multiply each stock return by its weight at the start of that interval.

r_i is a stock return and r_p is the portfolio return. Price subscripts 0 and 1 mean January 2 and 3; $W_0 = \text{USD } 1,000$. Initial weights are $w_i = 0.25$. Shares stay unchanged, without cash flows or fees. Each contribution $w_i r_i$ is shown in percentage points (pp).

$$r_p = \frac{\sum_i q_i (C_{i,1} - C_{i,0})}{W_0} = \sum_i w_i r_i$$

$$r_p \approx 0.25(-0.007488 - 0.000728 - 0.004358 + 0.008402) \approx -0.1043\%$$

Before running: Will the portfolio percentage return equal the sum or the average of these four stock percentage returns?

```
first_return = returns.iloc[0]
profit = shares * (close.iloc[1] - close.iloc[0])
contribution = weight * first_return * 100
pd.DataFrame({"Stock": tickers, "Return (%)": (first_return * 100).round(4),
              "Profit (USD)": profit.round(4),
              "Contribution (pp)": contribution.round(4)})
```

Stock	Return (%)	Profit (USD)	Contribution (pp)
AAPL	-0.7488	-1.8719	-0.1872
MSFT	-0.0728	-0.182	-0.0182
JPM	-0.4358	-1.0896	-0.109
XOM	0.8402	2.1004	0.21

The contributions sum to -0.1043 pp. Profit is USD -1.0431, so wealth becomes USD 998.9569. AAPL contributes -0.1872 pp, not its full -0.7488% stock return.

Discuss

1. Verify the portfolio return both from the four dollar profits and from the four contributions. Why do they agree?
2. After weights drift, may tomorrow's portfolio return still be calculated as the unweighted average of stock returns? Identify the weights required.

Sources: [1], [2]. See the references page.

ISSUE 03

Why do equal weights change without trading?

Equal weights do not remain equal by themselves. With unchanged shares, a stock that rises more than the portfolio becomes a larger fraction of the account.

V_i is a stock position's value (USD); W is total account value (USD). Here r_i and r_p cover January 2-31, with no trades or fees. Start and end refer to that same interval.

$$W_{i, \text{end}} = \frac{V_{i, \text{end}}}{W_{\text{end}}} = \frac{w_{i, \text{start}}(1 + r_i)}{1 + r_p}$$

$$W_{\text{AAPL}, \text{end}} \approx \frac{0.25(1 - 0.006680)}{1 + 0.020747} \approx 0.243283$$

Before running: Which stock will have the largest January 31 weight: the highest-priced stock, or the one with the largest price ratio since January 2?

```
jan_values = shares * close.loc["2024-01-31"]
jan_weights = jan_values / jan_values.sum()
pd.DataFrame({"Stock": tickers, "Start (%)": 100 * weight,
              "Jan 31 (USD)": jan_values.round(2),
              "Jan 31 (%)": (100 * jan_weights).round(2)})
```

Stock	Start (%)	Jan 31 (USD)	Jan 31 (%)
AAPL	25.0	248.33	24.33
MSFT	25.0	268.0	26.26
JPM	25.0	253.31	24.82
XOM	25.0	251.1	24.6

AAPL now has 24.33% rather than 25%. The total is still 100%, but the dollar amounts differ. There has been no new weighting decision: the change comes entirely from the observed prices.

Discuss

1. Using the displayed dollar values, verify the largest weight and calculate its percentage-point difference from 25%.
2. Would multiplying all four January 31 prices by the same positive factor change these weights? Show why using the numerator and denominator.

Sources: [1], [2]. See the references page.

ISSUE 04

What changes when only whole shares can be bought?

A small account may be unable to buy even one share of an expensive stock. Whole-share restrictions can therefore prevent exact equal weighting.

Use the same USD 250 budget per stock and zero fees. Floor means round down to the nearest whole share. This simple rule never spends more than a stock's budget; it is not a search for the closest feasible allocation. Unspent money remains cash.

$$q_i = \left\lfloor \frac{250}{C_i} \right\rfloor, \quad \text{cash} = 1000 - \sum_i q_i C_i$$

$$q_{\text{MSFT}} \approx \left\lfloor \frac{250}{370.87000} \right\rfloor = 0 \text{ shares}$$

Before running: Can USD 250 buy one MSFT share? Which holding will receive the unused money?

```
whole_shares = np.floor(250 / close.iloc[0])
whole_values = whole_shares * close.iloc[0]
cash_left = capital - whole_values.sum()
positions = whole_values.copy()
positions.loc["Cash"] = cash_left
pd.DataFrame({"Holding": positions.index,
              "Value (USD)": positions.round(2),
              "Weight (%)": (positions / capital * 100).round(2)})
```

Holding	Value (USD)	Weight (%)
AAPL	185.64	18.56
MSFT	0.0	0.0
JPM	172.08	17.21
XOM	204.72	20.47
Cash	437.56	43.76

MSFT receives no shares, and cash is USD 437.56, or 43.76% of the account. Renormalizing only the stock weights to 100% would hide this cash. Later issues return to fractional shares so that this rounding restriction does not change the comparisons.

Discuss

1. Under this per-stock budget rule, what minimum total capital lets the account buy at least one share of every stock at these prices?
2. If cash earns zero interest, how must it enter the next-day portfolio return? Explain why averaging only the three held stocks gives the wrong answer.

Sources: [1], [3]. See the references page.

ISSUE 05

When is a monthly rebalance executed?

A rebalance is a trade, not merely a new weight in a table. Specify when the rule is decided, when shares change, and which subsequent price movement the new shares earn.

Our monthly rule is fixed before trading: restore equal weights at the first observed Close of each month. The table labels the preceding Close as the decision date. A trade at today's Close cannot earn the price change that already ended at that Close.

Before running: If execution is March 1, will the first newly held return end on March 1 or the next observed trading date?

```
dates = all_close.index
months = pd.to_datetime(close.index).to_period("M")
monthly_dates = close.index[~months.duplicated()]
timing = []
for date in monthly_dates:
    j = dates.get_loc(date)
    timing.append([dates[j-1], date, dates[j+1]])
pd.DataFrame(timing, columns=["Decision", "Execution", "First return ends"])
```

Decision	Execution	First return ends
2023-12-29	2024-01-02	2024-01-03
2024-01-31	2024-02-01	2024-02-02
2024-02-29	2024-03-01	2024-03-04
2024-03-28	2024-04-01	2024-04-02
2024-04-30	2024-05-01	2024-05-02
2024-05-31	2024-06-03	2024-06-04

The February 29 decision is implemented at the March 1 Close. The new holdings first earn March 1-to-March 4 return. On March 1 itself, the old holdings earn the preceding interval before trading occurs.

We assume dollar orders can be allocated at the eventual Close with fractional shares. That is an idealized fill rule, not proof of an available broker service. The decision does not use the following return; the fill determines the share count.

Discuss

1. For the March 28 row, name the old-holding interval ending at execution and the first new-holding interval. Use the actual dates.
2. If a backtest applies the new April weights to the return ending April 1, which information-timing mistake does it make?

Sources: [1], [3]. See the references page.

ISSUE 06

Which holdings must be bought or sold to restore 25%?

To restore 25%, compare each current position with one quarter of the current account value. Sell positions above that amount and buy positions below it. Do not compare with the original USD 250.

At the February 1 Close, V_i is the value before trading, W is their total, and ΔV_i is the signed trade, all in USD. Positive means buy; negative means sell. Fractional shares and zero fees make the target $W/4$ feasible.

$$W = \sum_i V_i, \quad \Delta V_i = \frac{W}{4} - V_i$$

$$\Delta V_{\text{AAPL}} \approx \frac{1026.29769}{4} - 251.64297 \approx 4.93146 \text{ USD}$$

Before running: Which positions need sales, and should total purchases equal total sales when fees are zero?

```
pre_values = shares * close.loc["2024-02-01"]
pre_wealth = pre_values.sum()
target_value = pre_wealth / len(tickers)
trade = target_value - pre_values
pd.DataFrame({"Stock": tickers, "Before (USD)": pre_values.round(2),
              "Target (USD)": round(target_value, 2),
              "Buy / sell (USD)": trade.round(2)})
```

Stock	Before (USD)	Target (USD)	Buy / sell (USD)
AAPL	251.64	256.57	4.93
MSFT	272.18	256.57	-15.61
JPM	252.4	256.57	4.18
XOM	250.07	256.57	6.5

The new target is USD 256.57 per stock. Buy USD 4.93 of AAPL and sell USD 15.61 of MSFT. Purchases and sales each total USD 15.61; no profit is created. AAPL's February 1 Close is USD 186.86/share; divide its dollar trade by this price to find shares traded.

Discuss

1. Why would buying or selling back to USD 250 per stock fail to produce 25% weights at this date?
2. Which share counts change in this rebalance? Calculate the AAPL share change from its signed dollar trade and February 1 Close.

Sources: [1], [2], [4]. See the references page.

ISSUE 07

Which traded dollars receive the cost charge?

A proportional fee is charged on dollars traded. A rebalance can include both sales and purchases, and each side incurs a charge. Holding a stock without trading does not generate this fee.

Q is total bought plus sold value (USD), c is the assumed decimal fee rate, and F is the fee (USD). For Week 6 consistency, τ counts all traded stock dollars divided by wealth before trading, not half that sum.

$$Q = \sum_i |\Delta V_i|, \quad \tau = \frac{Q}{W}, \quad F = cQ$$

$$F \approx \frac{4}{10000} \times 31.21981 \approx 0.01249 \text{ USD}$$

Before running: Does charging only purchases miss half the rebalance's traded value in this zero-fee order calculation?

```
bought = trade.clip(lower=0).sum()
sold = -trade.clip(upper=0).sum()
traded = bought + sold
pd.DataFrame({"Event": ["Initial purchase", "Feb 1 rebalance"],
              "Bought (USD)": [capital, round(bought, 4)],
              "Sold (USD)": [0, round(sold, 4)],
              "Fee (USD)": [capital * .0004, round(traded * .0004, 4)]})
```

Event	Bought (USD)	Sold (USD)	Fee (USD)
Initial purchase	1000.0	0.0	0.4
Feb 1 rebalance	15.6099	15.6099	0.0125

Before the February 1 trades, W is USD 1026.30. Orders total USD 31.2198, so the estimated fee is USD 0.0125. Half the absolute weight changes would count only one side. At initial entry, all USD 1,000 is purchased; do not halve this stock-only amount.

These are fees for the orders from the zero-fee calculation. They do not yet fund those fees. Spending all USD 1,000 and then charging USD 0.40 would overdraw the account. The next issue reduces investment to pay the fee.

Discuss

1. Compute τ for the February 1 rebalance, and explain why using 4 bp times all account wealth overstates its fee.
2. If a report defines turnover as half the sum of absolute stock-weight changes during a fully invested rebalance, what factor is needed before applying a per-side cost rate? Why is initial entry different?

Sources: [1], [5]. See the references page.

ISSUE 08

How are fees paid before equal allocation?

Fees must come from the account. Invest slightly less than the available wealth, pay the charge, then divide the remaining investment equally. This is cash accounting, not a search for different weights.

Start with $W = \text{USD } 1,000$ cash. B is the total invested value after paying fees, in USD, not the remaining cash balance. The assumed fee rate is $c = 0.0004$. All invested dollars are purchases, so the fee is cB .

$$B + cB = W \implies B = \frac{W}{1 + c}$$

$$B = \frac{1000}{1.0004} \approx 999.60016, \quad F \approx 0.39984 \text{ USD}$$

Each of four stocks receives $B/4$, about USD 249.90004. The fee is about USD 0.39984, not USD 0.40, because it is charged on the amount actually purchased. Investment plus fee uses the original USD 1,000; no borrowing is needed.

This repeats Week 6's exact entry budget before extending it to a rebalance. Equal weighting applies to the money left to invest, not to the original cash before fees.

ISSUE 08 / CONTINUED

Pay for both purchases and sales

For this isolated February 1 calculation, use the fee-free positions from Issue 6 and no cash. These are not the running net account. V_i is each pretrade value, W their total, and B the total invested after fees, all in USD. Each new position should be $B/4$.

$$B + c \sum_i \left| \frac{B}{4} - V_i \right| = W$$

Stock	Before (USD)	Target (USD)	Trade (USD)
AAPL	251.6430	256.5713	+4.9283
MSFT	272.1843	256.5713	-15.6130
JPM	252.3971	256.5713	+4.1742
XOM	250.0733	256.5713	+6.4980

$$1026.28521 + 0.0004(31.21356) \approx 1026.29769 \text{ USD}$$

Investment is USD 1026.28521; the fee is USD 0.01249. Add the absolute trades, so sales also incur a fee. The target falls from the fee-free USD 256.57442 to USD 256.57130 per stock. Fees therefore change the orders as well as the remaining wealth.

ISSUE 08 / CONTINUED

Calculate the feasible investment budget

Try an investment amount between zero and available wealth. If investment plus its fee exceeds the account, lower that amount; otherwise raise it. Repeatedly halve the remaining interval. The function handles both entry and rebalancing; the output below checks the initial purchase against page 1 of this issue.

Before running: Will the initial invested amount be USD 1,000, USD 999.60, or slightly more than USD 999.60?

```
def investable_value(values, cash, rate):
    available = values.sum() + cash
    low, high = 0.0, available
    for _ in range(50):
        budget = (low + high) / 2
        fee = rate * abs(budget / len(values) - values).sum()
        if budget + fee > available:
            high = budget
        else:
            low = budget
    fee = rate * abs(low / len(values) - values).sum()
    return low, fee
entry_budget, entry_fee = investable_value(shares * 0, capital, .0004)
pd.DataFrame({"Available (USD)": [capital],
              "Invested (USD)": [round(entry_budget, 5)],
              "Fee (USD)": [round(entry_fee, 5)]})
```

Available (USD)	Invested (USD)	Fee (USD)
1000.0	999.60016	0.39984

Investment plus the USD 0.39984 fee equals USD 1,000, within numerical precision. With nonnegative holdings and $0 \leq c < 1$, the left side increases with B , so the budget has one solution. This function also handles later rebalances.

Discuss

1. Why is subtracting USD 0.40 first only an approximation here? Check the fee on the resulting purchase amount.
2. For the February 1 holdings, explain why fees change both the target dollars and the amounts bought or sold. Why must these be calculated together?

Sources: [1], [5]. See the references page.

ISSUE 09

How do we follow the account between rebalances?

An account needs shares and cash, not only a list of target weights. On each date, value yesterday's shares at today's Close. Only then execute any scheduled trade and pay its fee.

The function below uses the same four stocks and the budget calculation from Issue 8. Each output row is the account after any trade at that date's Close. Cash includes only any numerical remainder; no borrowing or new deposit is allowed.

Before running: Should the June 28 row have a fee if it is neither an entry nor a scheduled rebalance or liquidation?

```
def run_portfolio(rebalance_dates, rate):
    q = pd.Series(0.0, index=tickers)
    cash = capital
    rows = []
    for date, prices in close.iterrows():
        values = q * prices
        available = values.sum() + cash
        fee = 0.0
        if date in rebalance_dates:
            budget, fee = investable_value(values, cash, rate)
            q = budget / len(tickers) / prices
            cash = available - budget - fee
        wealth = (q * prices).sum() + cash
        rows.append([date, wealth, fee])
    return pd.DataFrame(rows, columns=["Date", "Wealth", "Fee"]).set_index("Date")
monthly = run_portfolio(monthly_dates, .0004)
shown = monthly.loc[["2024-01-02", "2024-02-01", "2024-06-28"]].round(5)
shown.columns = ["Wealth (USD)", "Fee (USD)"]
shown.insert(0, "Date", shown.index)
shown
```

Date	Wealth (USD)	Fee (USD)
2024-01-02	999.60016	0.39984
2024-02-01	1025.87486	0.01248
2024-06-28	1165.40253	0.0

Initial wealth after the entry fee is USD 999.60016; June 28 wealth is USD 1165.40253 with no terminal fee. The return from original capital is 16.5403%. Dividing by the already reduced January 2 wealth would omit the initial fee.

The closing-price assumption is unchanged. Dividends are excluded, so this is a price-only account, not an investor's complete realized return. The numerical remainder is below one billionth of a dollar for this run.

Discuss

1. If the account is actually liquidated on June 28, identify the additional trades and costs needed before calling its final value cash.
2. Which line ensures that old shares, not newly chosen shares, earn the price change ending on a rebalance date? Explain the order of calculations.

Sources: [1], [5]. See the references page.

ISSUE 10

Does rebalancing more often leave more money?

Rebalancing more often changes both future holdings and trading costs. Compare schedules using the same stocks, dates, initial cash, fill assumption and per-side fee rate. Do not attribute the entire wealth difference to fees.

Entry is always January 2. No rebalance occurs at the final Close. "None after entry" keeps the original shares; "Daily" restores equal weights at every intermediate Close. Gross uses zero fees; net uses 4 bp per dollar bought or sold.

Before running: Must the schedule with the most rebalances have the highest net ending wealth? Which columns would show that?

```
schedules = {"None after entry": close.index[:1],
            "Monthly": monthly_dates, "Daily": close.index[:-1]}
comparison = []
net_paths = {}
for name, dates in schedules.items():
    gross = run_portfolio(dates, 0)
    net = run_portfolio(dates, .0004)
    net_paths[name] = net["Wealth"]
    comparison.append([name, len(dates)-1, net["Fee"].sum(),
                      gross["Wealth"].iloc[-1], net["Wealth"].iloc[-1]])
comparison = pd.DataFrame(comparison, columns=["Schedule", "Rebalances",
      "Fees (USD)", "Gross (USD)", "Net (USD)"])
comparison.round(3)
```

Schedule	Rebalances	Fees (USD)	Gross (USD)	Net (USD)
None after entry	0	0.4	1159.936	1159.472
Monthly	5	0.476	1165.951	1165.403
Daily	122	0.807	1167.406	1166.497

Daily rebalancing ends at USD 1166.497, versus USD 1165.403 monthly. Paid fees are USD 0.807 and USD 0.476. Their gross results also differ, so costs alone do not explain the comparison.

These selected stocks and dates do not establish that any schedule wins in other markets. The same realized prices support only this stated historical comparison.

Discuss

1. Compare gross minus net wealth with total paid fees for the monthly schedule. Why need they not be identical?
2. To test another rebalance frequency, specify what must remain fixed and what later evaluation period you would reserve before seeing its returns.

Sources: [1], [4], [5]. See the references page.

ISSUE 11

Do equal dollar allocations have equal risk?

Equal money does not imply equally variable profits. A USD 250 position in a more volatile stock has a larger standard deviation of one-day dollar profit, even though its weight is also 25%.

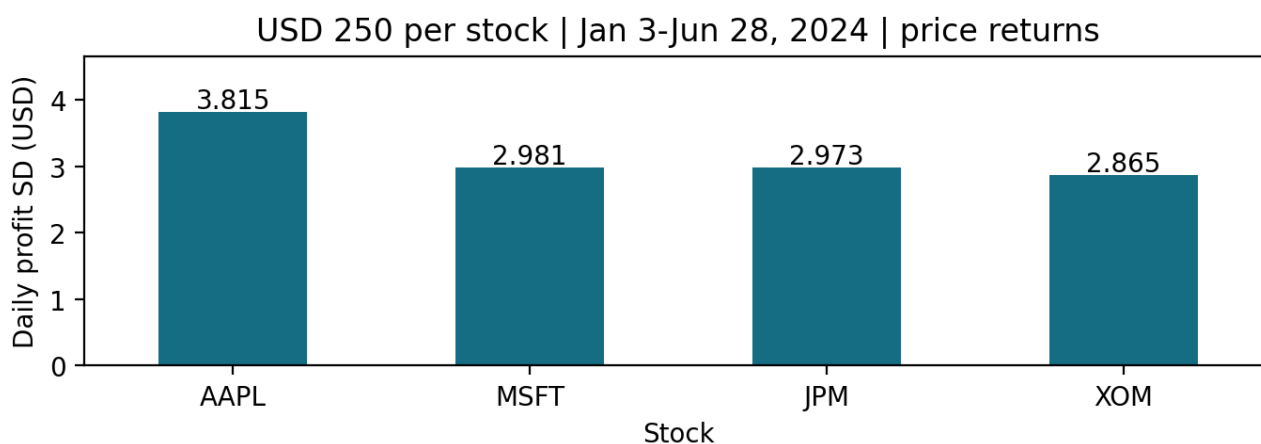
Use all 123 daily price returns from January 3-June 28. s_i is their sample standard deviation (decimal per day, ddof=1). For a hypothetical one-day starting allocation of USD 250, profit is $250r_i$. Scaling each return by 250 scales its standard deviation by 250.

$$SD(250r_i) = 250s_i$$

$$250s_{AAPL} \approx 250 \times 0.0152607 \approx 3.8152 \text{ USD}$$

Before running: Must the four bars have the same height because every allocation is USD 250?

```
daily_sd = returns.std(ddof=1)
dollar_sd = 250 * daily_sd
ax = dollar_sd.plot.bar(figsize=(7, 2.6), color="#156D83", rot=0)
ax.set(xlabel="Stock", ylabel="Daily profit SD (USD)",
       title="USD 250 per stock | Jan 3-Jun 28, 2024 | price returns")
for i, value in enumerate(dollar_sd):
    ax.text(i, value + .06, f"{value:.3f}", ha="center")
ax.set_ylim(0, dollar_sd.max() * 1.22)
plt.tight_layout()
```



AAPL gives USD 3.815 versus XOM USD 2.865. These are standalone daily-profit standard deviations at a fixed dollar exposure, not maximum losses and not additive contributions to portfolio risk.

This calculation does not follow the changing monthly account. It isolates the effect of stock-return variability while holding each starting dollar allocation fixed. Correlations are still needed to describe total portfolio volatility.

Discuss

1. For which stock is the displayed daily-profit standard deviation largest, and by how many USD does it exceed the smallest?
2. Would adding the four bar heights give portfolio-profit standard deviation? Identify the additional relationship between stock returns that is missing.

Sources: [1], [4], [5]. See the references page.

ISSUE 12

Does adding a stock necessarily reduce volatility?

More stock names do not guarantee lower volatility. Adding a stock changes every equal weight, and its returns may move together with existing holdings.

For two stocks, s is sample daily-return standard deviation (decimal) and ρ is sample correlation. M denotes MSFT and X denotes XOM. Squaring deviations of their half-and-half return produces two squared terms and a cross-product; the cross-product gives the covariance term $\rho s_M s_X$.

$$s_p^2 = \frac{s_M^2}{4} + \frac{s_X^2}{4} + \frac{\rho s_M s_X}{2}$$

Here $s_M \approx 0.0119248$, $s_X \approx 0.0114598$, and $\rho \approx -0.05872$. Substitute these same observations:

$$s_p \approx \sqrt{0.00003555 + 0.00003283 - 0.00000401} \approx 0.8023\%$$

Before running: When AAPL is added to MSFT and XOM, must the daily standard deviation fall?

```
groups = {"MSFT + XOM": ["MSFT", "XOM"],
          "Add AAPL": ["MSFT", "XOM", "AAPL"],
          "Add JPM": tickers}
volatility = []
for name, stocks in groups.items():
    equal_return = returns[stocks].mean(axis=1)
    volatility.append([name, len(stocks), 100 * equal_return.std(ddof=1)])
pd.DataFrame(volatility, columns=["Stocks held", "N", "Daily SD (%)"]).round(4)
```

Stocks held	N	Daily SD (%)
MSFT + XOM	2	0.8023
Add AAPL	3	0.8255
Add JPM	4	0.707

It rises from 0.8023% to 0.8255% when AAPL is added, then falls to 0.7070% with JPM. These cases were selected after examining the data to explain the calculation, not to select an investment universe.

Each row averages the listed stocks' returns over the same 123 intervals, with equal weights each day and no fees. It is not the monthly net account. This counterexample refutes "more names always means lower volatility"; it does not identify optimal weights.

Discuss

1. Why can the three-stock volatility exceed the two-stock volatility even though no stock has more than one-third of the money?
2. If a proposed weighting method claims lower risk than equal weighting, which universe, dates, cost rules and evaluation procedure must be fixed before comparing them?

Sources: [1], [4], [5]. See the references page.

REFERENCES AND REPRODUCTION

Sources and limits

[1] [yfinance](#): historical price fields

[2] [MSCI \(May 2021\): Equal Weighted Indexes Methodology](#), pp. 3-5

[3] [Investor.gov](#): fractional-share investing

[4] [Investor.gov](#): asset allocation, rebalancing and diversification

[5] [Lobo, Fazel and Boyd \(2007\)](#): linear and fixed transaction costs

The equal-weight definition is supported by [2], but our universe and monthly schedule are teaching choices. Source [5], Section 1 and Section 1.1, supports the budget, proportional-cost and covariance relationships; the displayed numerical examples and equal-weight budget derivation are our own. No optimization method from that paper is taught here. Read prices and formulas at full precision; displayed tables are rounded. Raw provider histories and calculation checks are retained privately.

Executed with Python 3.12.9, NumPy 2.2.6, pandas 2.2.3, matplotlib 3.10.8, yfinance 1.2.0. Run displayed code in order in a fresh Python session. Data requests need internet access; historical provider responses may change.

Selected cases illustrate the stated calculations, not investment recommendations or future-performance guarantees. Full snapshots and instructor checks are not included in the public PDF.